

Markerless 3D Human Pose Tracking

Roland Bernard, 19598, rolbernard@unibz.it

Tutor Oswald Lanz Domain Expert Marco Boschetti (Covision Lab)

Date September 2026

Abstract

The tracking of 3D human poses represents a crucial area in computer vision, with widespread applications across robotics, automated surveillance, healthcare, and entertainment. This project explores a tracking-by-detection framework designed to fuse multi-view 2D pose estimations from synchronized camera feeds into coherent, temporally consistent 3D skeletal trajectories. A system utilizes the YOLO26-pose model family for 2D keypoint extraction and extended Kalman filtering for 3D state estimation is developed, implemented, and evaluated. The resulting system was evaluated on eight video sequences from the CMU Panoptic dataset, benchmarking the impact of different system configurations on task performance and computational efficiency. Computational runtime analysis shows that the 2D detection phase dominates the processing, consuming between 64% and 91% of per-frame execution time and scaling linearly with the number of camera feeds. Furthermore, while overall Multi-Object Tracking Accuracy (MOTA) remains relatively stable at around 0.95 across configurations, increasing the viewpoint density improves 3D reconstruction precision. For the smallest model configuration, scaling from 4 to 16 cameras reduces the Mean Per-Joint Position Error (MPJPE) from 5.44 cm down to 5.44 cm. Similarly, increasing the backbone model capacity under a fixed camera count yields tracking error reductions.

1 Introduction

The estimation and tracking of 3D human poses is an important research area in computer vision, driven by its broad range of immediate applications across multiple domains. For example, in robotics and human-robot interactions it can find application in industrial environments and collaborative workspaces where robots must accurately perceive human poses and predict movements to maintain safe operational boundaries and avoid collisions with dangerous machinery. Similarly, for crowd control tracking of multi-person trajectories in public spaces facilitates anomalous behavior detection, crowd density planning, and automated surveillance. Other impactful applications can be found in healthcare, such as gait analysis, automated fall detection, and remote patient monitoring. Finally, human pose estimation and tracking can find applications also in the entertainment domain, such as the use real-time motion capture for virtual avatars or natural body-based control interfaces for gaming.

Despite the maturity of real-time 2D human pose estimation, transferring these methods to a robust and real-time

capable 3D setting remains a challenging task due to issues such as depth ambiguities, severe self-occlusions, and the high perspective variance of human motion. The primary objective of this project is to design, implement, and evaluate a computer vision architecture capable of tracking the skeletal movement of multiple people from 2D pose estimations extracted from multiple views by off-the-shelf 2D human pose estimation models. The system should be able to fuse the information gained from the 2D video frames of multiple camera viewpoints into coherent and temporally consistent 3D pose trajectories. For the evaluation of the result, this project will use a subset of the CMU Panoptic dataset (Joo, Liu, et al. 2015; Joo, Simon, et al. 2017), which contains videos captured from multiple calibrated cameras together with ground truth pose and tracking information for each frame. Using this evaluation set this project further aims to investigate the impact of model capacity for the 2D pose estimation model, as well as the impact of viewpoint count on both the achieved tracking accuracy and the runtime efficiency of the system. The remainder of this report is structured as follows: Section 4.1 describes the CMU Panoptic dataset that has been used for the evaluation of this project, explaining its structure and characteristics. Section 2 reviews the relevant theoretical background, referencing the course material where appropriate. Section 3 details the architecture of the tracking system, as well as the evaluation strategy including evaluation metrics. Section 5 presents the quantitative and qualitative evaluation results. Finally, Section 6 concludes the report with a summary of the findings and outlines limitations, highlighting potential areas for further improvements.

2 Background

This project builds directly upon concepts covered in the Computer Vision course (Lanz Spring 2026), particularly the chapters on Recognition and Object Tracking.

2.1 Object Detection and Pose Estimation

In object detection the goal is to localize all object, the number of which is not known a priori, in images (Lanz Spring 2026, p. 122-128). Like many computer vision recognition tasks, the field of object detection has evolved significantly with the introduction of deep learning (Lanz Spring 2026, p. 113). Deep learning-based object detection has evolved through two primary paradigms. Anchor-based models, like R-CNN or YOLO, rely on predefined bounding box priors of various scales and predicting offsets from these anchors.

In contrast, anchor-free models predict bounding boxes directly from point features or keypoint combinations. The main focus of the course, and also the method used in this project, fall under the anchor-based models. In the course we focused mainly on two stage detectors, specifically the R-CNN family of model (Lanz Spring 2026, p. 125-127). In these models, there are two separate inference stages. The first stage generates a large number of candidate regions and the second stage then classifies these regions. Fast R-CNN (Girshick 2015) and Faster R-CNN (Ren et al. 2015) improve inference speed over the original R-CNN by using a shared feature backbone for both stages and by using a Region Proposal Network to create candidate regions.

In contrast to these two-stage detectors, the YOLO model used in this project is a single stage detector. It works by dividing the input image into multiple grids of different resolutions. Then a CNN based architecture is employed to extract features for each of the grid positions. Finally, one or more prediction head are applied to the resulting feature maps to simultaneously predict both object presence and bound box offsets from grid location defined anchors. The main advantage of single-stage approaches is that they are usually faster than two-stage approaches, although they often perform worse in practice (Lanz Spring 2026, p. 126-127).

Once an object is detected, one can append extra heads to the feature maps to also predict other properties about the objects. An example of this approach is discussed in the course through the Mask R-CNN architecture (K. He et al. 2017) which predicts instance masks for each detected object (Lanz Spring 2026, p. 128). However, this approach can also be applied to Human Pose Estimation, where the additional output head of the network is instead used to predict keypoint locations (Maji et al. 2022). This approach is taken by the YOLO26 models used in this project (Jocher G. 2026; Sapkota, Cheppally, et al. 2025; Sapkota and Karkee 2025).

2.2 Object Tracking

Detecting objects or estimating poses in individual frames is insufficient for many applications (Lanz Spring 2026, p. 129). Instead, what is often required is to associate detections with the physical objects they belong to and model how those evolve over time. This problem is called object tracking. One approach to tackle this problem is tracking-by-detection, and it can be divided into three essential steps, detection, association, and filtering. In detection, candidate objects are detected in individual frames. Then, these detections are assigned to concrete objects in the association step. Finally, filtering is used to determine the most likely state of the object given the associated detections. Filtering is responsible for refining the noise inherently present in the detection using probabilistic models and preceding observations. While the detection step has already been discussed in Section 2.1, we will now focus on the association and filtering steps.

In many application multiple targets must be tracked at the same time. To understand which detection belongs to

which underlying object, association must be performed (Lanz Spring 2026, p. 134-135). A typical way to proceed it to predict object from the previous frame and to use those predictions to associate detections to the objects. This association can be performed using an association measure, which is a measure indicating the distance or similarity between a predicted object track and a detection. A naive approach, and also the one applied in this project, is to measure the overlap of the prediction and the detection (Bewley et al. 2016). Alternative approaches, as outlined in (Lanz Spring 2026, p. 135-136) can include color histograms or optical flow and measure agreement. It is also possible to used metric learning, where a deep learning model is used to extract features for computing the association measure.

After an association measure has been established, it is still necessary to actually perform the association (Lanz Spring 2026, p. 136-140). A naive approach can be to consider all detections within some gating area of the prediction and then associate the closest of these detections to the prediction. The distance in this context can also consider the probabilistic model of the prediction to find most likely detections. However, this nearest neighbor approach can easily fail due to ambiguities. A better approach it to use a bipartite graph matching based framework. In this framework, the problem is modeled as a bipartite graph from predictions to detections, with edges weighted by the association metric. This can be formulated as the following integer linear programming problem:

$$\begin{aligned} \underset{\mathbf{Z}}{\operatorname{argmax}} \quad & \sum_{i=1}^M \sum_{j=1}^N s_{ij} z_{ij} \\ \text{s.t.} \quad & \forall j : \sum_{i=1}^M z_{ij} = 1 \quad \forall i : \sum_{j=1}^N z_{ij} = 1 \quad z_{ij} \in \{0, 1\} \end{aligned}$$

where s_{ij} and z_{ij} are respectively the association metric and association between prediction i and detection j . This problem can then be solved either using a greedy algorithm, or as has been done for this project, using globally optimal algorithms like the Hungarian method (Kuhn 1955). Finally, another possible approach to association is to used advanced graph-based deep learning networks.

After detections and objects have been associated, it is still necessary to perform the filtering step (Lanz Spring 2026, p. 130-134). It is crucial to understand that detections differ from the underlying state that we try to understand. Aside from being noisy, detections might also be missing some information of interest in the object. For example, object detections usually do not include velocity, and still we would like to infer it from the detections. When represented as using a Markov model, the state of a target at any given time step depends only on that targets state in the previous time step. Further, the observation or detection at any time step depends only on the targets current state. One way to evaluate this process is using recursive Bayesian estimation, also called Bayesian filtering.

In Bayesian filtering is applied in two phases. First, the prediction step is used to model the target states distribution using only the past observations. Due to the conditional

independence assumption of the hidden Markov model, this probability distribution of the hidden state can be computed as followed:

$$\begin{aligned} p(\mathbf{x}_t | \mathbf{z}_{1:t-1}) &= \int p(\mathbf{x}_t, \mathbf{x}_{t-1} | \mathbf{z}_{1:t-1}) d\mathbf{x}_{t-1} \\ &= \int p(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{z}_{1:t-1}) p(\mathbf{x}_{t-1} | \mathbf{z}_{1:t-1}) d\mathbf{x}_{t-1} \\ &= \int p(\mathbf{x}_t | \mathbf{x}_{t-1}) p(\mathbf{x}_{t-1} | \mathbf{z}_{1:t-1}) d\mathbf{x}_{t-1} \end{aligned}$$

where $p(\mathbf{x}_t | \mathbf{x}_{t-1})$ defines out motion model, and $p(\mathbf{x}_{t-1} | \mathbf{z}_{1:t-1})$ is the posterior distribution after applying all previous observations. After prediction, we want to incorporate the information gained by the new observation. This can be achieved as follows by applying Bayes' rule:

$$p(\mathbf{x}_t | \mathbf{z}_{1:t}) \propto p(\mathbf{z}_t | \mathbf{x}_t) p(\mathbf{x}_t | \mathbf{z}_{1:t-1})$$

where $p(\mathbf{z}_t | \mathbf{x}_t)$ models the observation process.

The equations of the Bayesian filtering algorithm can be implemented by representing the probability distributions as each time stem in parametric form, such as in Kalman filtering (Lanz Spring 2026, p. 132-134) (Bar-Shalom et al. 2001), or in nonparametric form, such as with particle filters. In a linear Kalman filter the motion and observation models are defined as:

$$\begin{aligned} \mathbf{x}_t &= \mathbf{F}_t \mathbf{x}_{t-1} + \mathbf{B}_t \mathbf{u}_t + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_t) \\ \mathbf{z}_t &= \mathbf{H}_t \mathbf{x}_t + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_t) \end{aligned}$$

The optimal closed-form solution is the standard Kalman Filter, which updates the mean $\bar{\mathbf{x}}_t$ and covariance Σ_t of $\mathbf{x}_t$ through linear projection and gain computation.

While the linear Kalman filter is applicable in cases where both the motion and observation model are linear, in our application of multi-view tracking, the measurement model is non-linear because 3D world coordinates must project onto a 2D camera plane. For this reason, this project makes use of extended Kalman filtering. We define the observation function to be $\mathbf{z}_t = h(\mathbf{x}_t) + \mathbf{v}_t$. The extended Kalman filter linearizes this non-linear function using a first-order Taylor expansion around the predicted state estimate $\bar{\mathbf{x}}_t$:

$$h(\mathbf{x}_t) \approx h(\bar{\mathbf{x}}_t) + \mathbf{J}_h(\mathbf{x}_t - \bar{\mathbf{x}}_t)$$

where $\mathbf{J}_h$ is the Jacobian matrix evaluated at $\bar{\mathbf{x}}_t$. In the update step the Jacobian matrix is the used in place of the linear observation matrix $\mathbf{H}_t$ to update the covariance and calculate Kalman gain.

2.3 Projective Geometry

The dataset used in this project already contains camera calibration parameters for all camera views. Still, understanding the meaning of these parameters and how they relate to the image formation process is critical for the correct implementation for the triangulation algorithm as well as for modeling the observation process. The mapping of 3D points to 2D pixels is defined mathematically through

the pinhole camera model (Lanz Spring 2026, p. 6-8) (Hartley et al. 2003). Unlike a physical pinhole camera, which would invert the image, the mathematical model places the image plane in front of the focal point for convenience. The full perspective projection model linking a homogenous 3D world coordinate $\tilde{\mathbf{x}}_w$ to a homogenous 2D sensor pixel $\tilde{\mathbf{x}}_s$ is given by:

$$\tilde{\mathbf{x}}_s = \mathbf{K}[\mathbf{R}|\mathbf{t}]\tilde{\mathbf{x}}_w = \mathbf{P}\tilde{\mathbf{x}}_w$$

where $\mathbf{P}$ is the full 3×4 projection matrix. It decomposes into intrinsic parameters $\mathbf{K}$ and extrinsic parameters $\mathbf{R}$ and $\mathbf{t}$. For a perspective projection, 3D points $\tilde{\mathbf{x}}_s$ in the local camera coordinate system are mapped to the image plane by dividing them by their depth Z component.

The assumption of linear projection made by the above pinhole camera model, where straight lines remain straight, is violated by physical camera lenses, introducing distortions (Lanz Spring 2026, p. 8). These must be modeled to accurately project points or undistort pixel coordinates. Two types of distortions are typically modeled. First, radial distortion causes points to bend outward inward. It is modeled relative to the squared radius $r^2 = x^2 + y^2$. Second, tangential distortion arises when the lens is not perfectly parallel to the imaging sensor.

2.4 Multi-View Triangulation

Even once corresponding points across multiple camera views have been identified, their 3D position $\tilde{\mathbf{x}}_w$ must still be reconstructed from the 2D pixel locations. Rays back-projected from pixels will nearly never intersect perfectly in 3D space due to measurement noise. As such, a noise robust algorithm must be used that can recover 3D coordinates using multiple noise 2D projections (Lanz Spring 2026, p. 16-17) (Hartley et al. 2003). If we denote $\tilde{\mathbf{x}}_i^s$ the homogeneous 2D observation in the i -th camera, and $\mathbf{P}_i$ the projection matrix of that that camera, because $\tilde{\mathbf{x}}_i^s$ and the projected point $\mathbf{P}_i \tilde{\mathbf{x}}_w$ represent the same point up to scale, their cross product must be zero:

$$\tilde{\mathbf{x}}_i^s \times \mathbf{P}_i \tilde{\mathbf{x}}_w = 0$$

This cross product yields a set of three linear equations for each camera view. However, the third is a linear combination of the first two, so it can be discarded. By stacking the equations from $N \geq 2$ camera views, we obtain a homogeneous linear system $\mathbf{A}\tilde{\mathbf{x}}_w = 0$. Because $\tilde{\mathbf{x}}_w$ is defined only up to scale, we impose the constraint $\|\tilde{\mathbf{x}}_w\| = 1$. The optimal least-squares solution to this system is found by computing the Singular Value Decomposition of $\mathbf{A}$ and taking the right singular vector associated with the smallest singular value. Alternatively, it is also possible to transform the homogeneous system in to an in-homogeneous system and solve it using standard least squares. This latter approach is taken in the project.

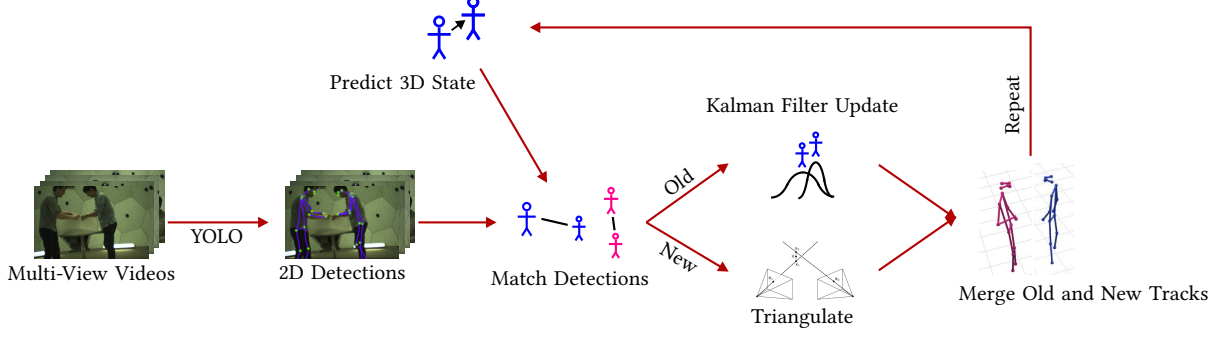


Figure 1: Architecture overview of the developed tracking system. Images are passed through a 2D pose estimation model to obtain the detections. These are matched against the predictions and the other cameras detections. New tracks are initialized using triangulation, and old tracks updated using Kalman filter update.

3 Methodology

This section of the report will go into detail about the internal implementation of the tracking system.

3.1 Architecture Overview

Figure 1 show the overall pipeline used in this projects tracking algorithm. The core of the tracking pipeline runs a recursive estimation loop on incoming multi-view video frames. The input to the system is a number of synchronized multi-view video feeds, and the final result is a 3D representation of the skeletal joint of each person in the scene.

First, multi-view videos are passed through the YOLO detection network, yielding 2D bounding boxes and keypoints. Concurrently, the tracking module predicts the current 3D state of all previously known tracks based on continuous physics models. The keypoints of the predicted tracks are projected into the 2D camera planes and matched against the newly observed 2D detections. Detections matched to existing tracks trigger an extended Kalman filter update for the matched track to refine its 3D state. Unmatched, novel 2D detections matched between views, and if at least two views contain the new target, triangulated to instantiate new 3D tracks. Finally, updated tracks and newly created tracks are merged, and the loop repeats for the next timesamp.

3.2 2D Detections and Uncertainty Estimation

During inference, images from each active camera view are batched and passed to the YOLO26-pose detector using PyTorch (Paszke et al. 2019; Ansel et al. 2024). The detector outputs 2D keypoint positions along with visibility confidence scores $c_k \in [0, 1]$. As a first filter we select only detections that have a confidence above τ_{obj} . Further, we filter out any detections for which fewer than N_{kpt} are visible with a confidence above τ_{kpt} . However, for performing the Kalman filter updates we need a probabilistic framework in which we model the prediction uncertainty as Gaussian noise. This need can not be satisfied by the confidence

score alone. To estimate the measurement noise covariance dynamically from the outputs of the YOLO model, the system relies on bounding box and visibility scaling. The variance σ_k^2 for each detected keypoint k is computed dynamically based on the bounding box magnitude and the joint visibility confidence. The specific formulation used is:

$$\sigma_k^2 = \sigma_{min}^2 + d_{bb}^2 \cdot \left(\frac{\sigma_{vis}^2}{\max(0, \bar{c}_k) + \frac{\sigma_{vis}^2}{\sigma_{inv}^2}} \right)$$

where $\bar{c}_k = \frac{c_k - \tau_{kpt}}{1.0 - \tau_{kpt}}$ is the confidence score normalized above the tuned keypoint threshold τ_{kpt} , d_{bb}^2 is the length of the diagonal of the bounding box, and σ_{min} , σ_{vis} , σ_{inv} are tuning parameters specifying respectively the minimum variance, minimum bounding box dependent variance, and maximum bounding box dependent variance. This ensures that occluded joints are appropriately penalized with high uncertainty while visible joint get a lower variance. The keypoint detections are assumed to all be independent, so the final measurement noise covariance matrix is build by filling the diagonals.

Further, before passing the resulting 2D keypoints to the rest of the tracking system, the 2D pixel coordinates are transformed into normalized camera coordinates utilizing the camera intrinsics and distortion coefficients as follows:

$$\mathbf{x}_{pixel} = \begin{bmatrix} u \\ v \end{bmatrix} \xrightarrow{K^{-1}} \mathbf{x}_{distorted} \xrightarrow{\text{Iterative Undistortion}} \mathbf{x}_{normalized}$$

To maintain consistent uncertainty representation, we also transform the observation covariance matrix Σ_{pixel} for the 17 keypoints into normalized space using the following approximation:

$$\Sigma_{norm} \approx K^{-1} \Sigma_{pixel} K^{-T}$$

The main motivation behind this approach is that it simplifies the triangulation for new tracks as well as the observation process modeling in the Kalman filter update for old tracks by removing the non-linear distortion effects. As an additional benefit, it becomes more efficient to compute not only the Kalman filter updates but also the computation of cost matrices for the association step, which need to perform projection from the internal 3D state.

3.3 State Representation and Kalman Filtering

In the Kalman filter implementation of the project, the state for each person is tracked separately. The state vector is composed by modeling the human pose as a collection of 17 keypoints. The state vector $\mathbf{x} \in \mathbb{R}^{102}$ encapsulates the 3D coordinates and the 3D velocities for all 17 keypoints resulting in a state vector shaped as:

$$\mathbf{x} = [\mathbf{p}_1^T, \dots, \mathbf{p}_{17}^T, \mathbf{v}_1^T, \dots, \mathbf{v}_{17}^T]^T$$

where $\mathbf{p}_k = [x_k, y_k, z_k]^T \in \mathbb{R}^3$ and $\mathbf{v}_k = [\dot{x}_k, \dot{y}_k, \dot{z}_k]^T \in \mathbb{R}^3$ are the 3D position and velocity vectors for keypoint k .

The state dynamics are modeled by a continuous-time linear velocity process. The decision was made to specify the dynamics in the continuous domain in order to be able to handle variable frame rates which could for example arise due to frames being dropped during computationally more demanding scenes. The continuous dynamics matrix $\mathbf{F}$ and process noise $\mathbf{Q}$ are mapped to discrete time steps Δt_i dynamically for each frame i using a second-order Taylor expansion based approximation:

$$\begin{aligned}\mathbf{F}_k &\approx \mathbf{I} + \mathbf{F}\Delta t_i + \frac{1}{2}\mathbf{F}^2\Delta t_i^2 \\ \mathbf{Q}_k &\approx \mathbf{Q}\Delta t_i + \frac{1}{2}(\mathbf{F}\mathbf{Q} + \mathbf{Q}\mathbf{F}^T)\Delta t_i^2\end{aligned}$$

The measurement function $h_c(\mathbf{x}) : \mathbb{R}^{102} \rightarrow \mathbb{R}^{34}$ acts as a non-linear pinhole projection mapping the 102-dimensional 3D state back to a 34-dimensional observation of 2D coordinates in camera c :

$$h_c(\mathbf{x}) = [\pi_c(\mathbf{p}_1)^T \quad \pi_c(\mathbf{p}_2)^T \quad \dots \quad \pi_c(\mathbf{p}_{17})^T]^T$$

where $\pi_c(\mathbf{p}_k)$ is the perspective pinhole projection of the 3D point into normalized camera coordinates:

$$\pi_c(\mathbf{p}_k) = \frac{1}{Z_{c,k}} \begin{bmatrix} X_{c,k} \\ Y_{c,k} \end{bmatrix} = \frac{1}{\mathbf{R}_{c,3}\mathbf{p}_k + t_{c,3}} \begin{bmatrix} \mathbf{R}_{c,1}\mathbf{p}_k + t_{c,1} \\ \mathbf{R}_{c,2}\mathbf{p}_k + t_{c,2} \end{bmatrix}$$

Because multiple cameras observe the target simultaneously, the system concatenates these independent multi-camera views into a single centralized extended Kalman filter observation update instead of applying them sequentially. For a track observed by M cameras the observations are merged as follows:

$$\begin{aligned}\mathbf{z}_{merged} &= [\mathbf{z}_1^T : \mathbf{z}_M^T]^T \in \mathbb{R}^{34M} \\ \Sigma_{merged} &= \text{diag}(\Sigma_1, \dots, \Sigma_M) \in \mathbb{R}^{34M \times 34M}\end{aligned}$$

This formulation can handle asynchronous data naturally by simply omitting cameras without active detections.

Because the projection mapping is non-linear, the extended Kalman filter requires the computation of the Jacobian $\mathbf{H}_i \in \mathbb{R}^{34M \times 102}$. Rather than deriving and hard-coding this computation analytically, the system leverages the automatic differentiation capabilities of PyTorch (Paszke et al. 2019; Ansel et al. 2024) to compute the exact measurement Jacobian dynamically at runtime.

3.4 Data Association

Data association is performed in two separate steps. First, the detections for in view are matched against the existing 3D track predictions. This is performed independently for each view. Next, all detections that were not matched to any of the existing tracks must be matched between views in order to find opportunities for triangulation.

The association between detections and existing tracks is performed independently for each view by first projecting the prediction to the 2D camera plane. Note that we also project the 3D covariances using the first-order approximation using the Jacobian. A cost matrix is populated between projected predictions and YOLO derived detections. The distance metric is a modified Mahalanobis distance that adds the log-determinant of the combined covariance to discourage highly uncertain matches with low statistical power:

$$\begin{aligned}d_{\Sigma}(\mathbf{z}, \hat{\mathbf{z}}) &= (\mathbf{z} - \hat{\mathbf{z}})^T \Sigma^{-1} (\mathbf{z} - \hat{\mathbf{z}}) \\ C_{i,j} &= d_{\Sigma_{pred} + \Sigma_{obs}}(\mathbf{z}, \hat{\mathbf{z}}) + \log |\Sigma_{pred} + \Sigma_{obs}| - N_{dim}\theta_{mo}\end{aligned}$$

where $N_{dim} = 34$, and θ_{mo} is a threshold parameter. Note that we fill the cost matrix with zero padding to allow for unmatched tracks or detections. The optimal association is found by solving this cost matrix using the Hungarian algorithm (Kuhn 1955) via the SciPy library (Virtanen et al. 2020) implementation. This process can be interpreted as an implementation of maximum likelihood based model selection amongst all possible associations.

Detections that are not matched to existing tracks by the above method may belong to new people entering the scene. Since these detections lack 3D track priors, we must match them across different camera views. To achieve this goal, this project employs an iterative greedy-optimal matching approach to associate 2D detections across views. The algorithm works by first building associations between the first two views, then iteratively adding views. During the iterative addition step we compute a cost matrix and solve for the optimal association using the Hungarian algorithm in an effort to minimize the reprojection error:

$$C_{cross} = \frac{1}{K} \sum_{k=1}^K (d_{\Sigma_k}(\mathbf{z}_k, \pi_k(\mathbf{X}_{3D})) + \ln |\Sigma_k|) - N_{dim}\theta_{mn}$$

where $\mathbf{X}_{3D}$ is the triangulated 3D point from the K associated views, and θ_{mn} is the new-track association threshold. Detections matched in at least two views are grouped to initialize a new track via triangulation.

3.5 Track Creation and Deletion

To prevent spurious detections from creating false tracks, we manage the lifecycle of each track using a state machine. New tracks that have just been added via triangulation of detections start out as tentative tracks. If a tentative track is not matched to at least one detection in the next frame, it is deleted immediately. Once a tentative track have been observed for $\geq N_{age}$ consecutive frames, it is promoted to an active tracks. Active tracks are more robust to occasional

missing detections. If an active track is not observed, its state is updated only using the Kalman filter prediction step. However, if it remains unmatched for more than N_{inv} consecutive frames, it is deleted.

3.6 Skeletal Constraints

3.7 Learning Parameters from Data

3.7.1 Custom YOLO Output Head

3.7.2 Learning the Kalman Filter

3.7.3 Tuning Other Parameters

4 Experimental Evaluation

This section of the report will focus on how the proposed system has been experimentally evaluated. Further it covers the dataset used for the experiments and the selected evaluation metrics.

4.1 Dataset Description

For the development and evaluation of the tracking system in this project, the CMU Panoptic Dataset (Joo, Liu, et al. 2015; Joo, Simon, et al. 2017; Simon et al. 2017) has been used. This is a large-scale dataset that was originally recorded to facilitate research in multi-view social motion capture. Figure 2 shows a sample of some still images extracted from the video sequences contained in this dataset.

The complete dataset includes 84 video sequences totaling approximately 11.5 hours of video captured in a specialized 5.49m diameter dome-shaped room equipped with inward facing cameras attached to the outside panels. In total, the recording hardware consists of 511 calibrated cameras, 480 VGA cameras recording at a 640×480 resolution at 25 fps and 31 HD cameras recording at a 1920×1080 resolution at 30 fps. The room also contains 10 Kinect II (Kurillo et al. 2022) sensors and 5 DLP projectors. Because of the geometry of the room and the placement of cameras, no single camera is able to view the complete space. This means that while each point in the space is visible from multiple cameras, each camera only captures a partial view of the environment, introducing occlusion and field-of-view limits related challenges for the tracking task.

The dataset also includes ground truth 3D skeletons and tracks that were generated by the CMU researchers using algorithms that leverage the entirety of the camera array (Joo, Liu, et al. 2015). Ground truth annotations of the dataset are provided in the COCO19 format. To align them with the outputs of the YOLO-based pose estimation networks used in this project (Jocher G. 2026), the ground truth data was pre-processed by converting it into the COCO17 format (T.-Y. Lin et al. 2014). This was achieved by discarding the two extra joint positions. The dataset also includes the calibration parameters for all available camera views, including camera extrinsics, intrinsics, and distortion parameters.

Due to computational and time constraints, running the tracking system across all the sequences for the dataset

was not feasible. For this reason, a subset of the dataset was selected. Five video sequences were used for the evaluation, namely 171204_pose6, 161029_piano4, 170915_office1, 161029_build1, and 160224_haggling1. Furthermore, evaluation was performed using only a subset of all camera views. A maximum of 16 VGA-type cameras were utilized per video sequence. These 16 cameras were selected based on the official dataset download script¹, which ensures they are approximately uniformly distributed around the dome to provide maximum spatial coverage with a minimal camera count.

4.2 Evaluation Procedure

For this project the system has been evaluated by comparing its 3D tracking predictions frame-by-frame against the ground truth annotations in the CMU Panoptic dataset. The experiments were run across all five selected sequences mentioned in Section 4.1, evaluating various combinations of YOLO pose estimation models, testing YOLO26n-pose, YOLO26s-pose, YOLO26m-pose, and YOLO26l-pose, and spacial view densities of 4, 8, 12, and 16. Note that due to computation demands, only the YOLO26n-pose model has been tested with more than 4 cameras. All evaluations were performed on AMD Ryzen AI 9 HX 370 CPU with most PyTorch (Paszke et al. 2019; Ansel et al. 2024) operation running on the integrated AMD Radeon 890M GPU.

Due to the scope of the project, system parameter of the system were initialized heuristically rather than via exhaustive grid-search or end-to-end learning. While these rigid parameter values are functional, an identified area for improvement is optimizing them directly from tracking data either using grid search or end-to-end learning.

Detection thresholds for the YOLO based detector we set to $\tau_{obj} = 0.5$ and $\tau_{kpt} = 0.25$, using also $N_{kpt} = 3$ as the minimum number of visible keypoints. The minimum pixel variance has been set to $\sigma_{min}^2 = 16.0$ pixels², with visibility and invisibility scale variances set to $\sigma_{vis}^2 = 0.005$ and $\sigma_{inv}^2 = 5$. The matching thresholds for both cross-view and prediction-view matching have both been set to $\theta_{mn} = \theta_{mo} = 0$. For the tracking lifecycle we configured the age for tentative track promotion to $N_{age} = 3$ and the maximum duration without detections for active tracks to $N_{inv} = 10$. Finally, the continuous physic matrices have been set as follows:

$$\mathbf{F} = \begin{bmatrix} \mathbf{0}_{51 \times 51} & \mathbf{I}_{51 \times 51} \\ \mathbf{0}_{51 \times 51} & -0.5 \cdot \mathbf{I}_{51 \times 51} \end{bmatrix}, \mathbf{Q} = \begin{bmatrix} \sigma_p^2 \cdot \mathbf{I}_{51 \times 51} & \mathbf{0}_{51 \times 51} \\ \mathbf{0}_{51 \times 51} & \sigma_v^2 \cdot \mathbf{I}_{51 \times 51} \end{bmatrix}$$

with $\sigma_v^2 = (2\frac{m}{s})^2$ and $\sigma_p^2 = (5cm)^2$. This matrix models constant velocity dynamics with a damping term of -0.5 on velocity to stabilize motion predictions.

4.3 Evaluation Metrics

To measure tracking and reconstruction performance, this project makes use of standard metrics from the Multi-Object

¹<https://github.com/CMU-Perceptual-Computing-Lab/panoptic-toolbox/blob/master/scripts/getData.sh>

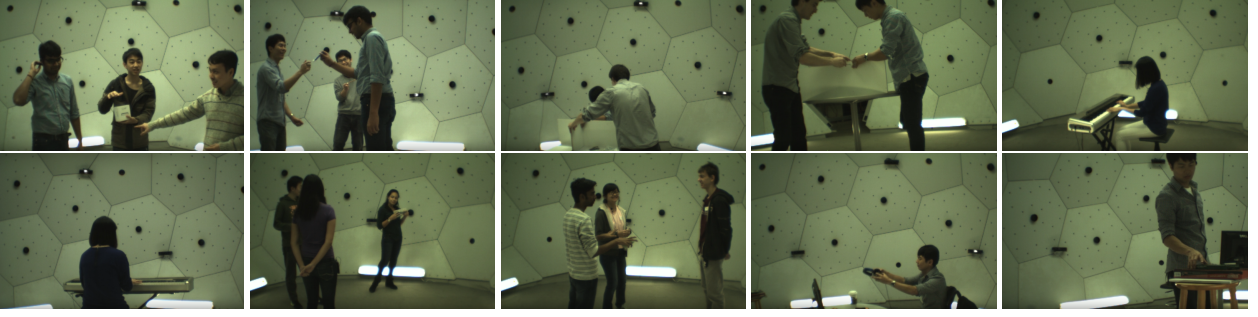


Figure 2: A series of example images extracted from the video sequences of the CMU Panoptic dataset.

Tracking and 3D Pose Estimation literature (Bernardin et al. 2008; Ionescu et al. 2013). All measures are reported in centimeters, and we use a mean joint error of 15 centimeters as the threshold for matches for all metrics except MPJPE.

Mean Per-Joint Position Error (MPJPE) MPJPE measures the average 3D Euclidean distance between predicted and ground truth keypoints across all matched frames and joints.

$$\text{MPJPE} = \frac{1}{N_{\text{matched}}} \sum_{i=1}^{N_{\text{matched}}} \left(\frac{1}{17} \sum_{k=1}^{17} \|\mathbf{p}_{k,i}^{\text{pred}} - \mathbf{p}_{k,i}^{\text{gt}}\|_2 \right)$$

For this metric we do not apply a matching threshold, instead matching all predictions against the best matching ground truth track via Hungarian method based assignment.

Multi-Object Tracking Accuracy (MOTA) MOTA evaluates overall tracking performance by accounting for three types of errors, false positives, false negatives, and identity switches:

$$\text{MOTA} = 1.0 - \frac{\sum_t \text{FN}_t + \text{FP}_t + \text{IDS}_t}{\sum_t \text{GT}_t}$$

where FN_t is the number of false negatives, FP_t the number of false positives, IDS_t that of identity switches, and GT_t is the number of ground truth individuals in frame t .

Multi-Object Tracking Precision (MOTP) MOTP measures the alignment of tracked objects by calculating the average distance error for correctly matched targets. Note that in contrast to MPJPE this metric is only computed over the true positives matched within the 15 centimeters maximum threshold.

$$\text{MOTP} = \frac{\sum_{t,d} \frac{1}{17} \sum_{k=1}^{17} \|\mathbf{p}_{t,d,k}^{\text{pred}} - \mathbf{p}_{t,d,k}^{\text{gt}}\|_2}{\sum_t \text{TP}_t}$$

where TP_t is the number of true positives.

Precision and Recall Finally, we also report the standard precision and recall metrics.

$$\text{Precision} = \frac{\sum_t \text{TP}_t}{\sum_t \text{TP}_t + \text{FP}_t}, \quad \text{Recall} = \frac{\sum_t \text{TP}_t}{\sum_t \text{TP}_t + \text{FN}_t}$$

5 Results and Discussion

This section of the report will present the evaluation results obtained by the developed system on the chosen dataset. It further contains also discussion of the presented results and possible reasons for the observed deficiencies.

5.1 Runtime Efficiency

Table 1 shows the mean per-frame runtime of the developed tracking system across the selected test sequences for different camera count and model capacity configurations. It can be seen that across every single configuration, the detection step consumes the vast majority of the processing time. For the base setup with YOLO26n-pose and 4 camera vies, detection takes 0.0445 seconds out of a 0.0697-second total which equals approximately 64% of the frame time. This discrepancy becomes wider when increasing the model capacity. For the larges model, YOLO26l-pose, still with only 4 cameras, the detection takes up to 0.2613 seconds out of a total of 0.2879 seconds, which represents approximately 91% of the time.

When adding more camera views to the system using the smallest model, the computational load predictably increases. Because the detection model has to run a forward pass on every single camera feed, detection time scales almost perfectly linearly with the number of cameras, from 0.045s for 4 cameras to 0.181s for 16. Steps like matching and EKF updates also scale up as camera counts increase because these need to handle more cross-camera data points to associate and filter. However, their growth is somewhat slower than the linear increase for detection. The EKF prediction step is largely a negligible part of the overall system runtime.

When keeping the camera count at 4 but upgrading the YOLO architecture to larger capacity version, it can be observed that the cost increases significantly. Upgrading from YOLO26n-pose to YOLO26s-pose doubles the detection time, going from 0.0445s to 0.0876s. However, jumping to the medium or large version causes a massive spike, tripling the time to over a quarter of a second. Predictably, only the runtime for the detection step is affected by increasing the model capacity and matching, EKF update, and Prediction remain almost entirely unchanged across all four model variants.

With the fastest configuration tested, the system achieves

Model	# Cameras	Detection	Matching	EKF Update	EKF Prediction	Total
YOLO26n-pose	4	0.0445	0.0166	0.0080	0.0005	0.0697
	8	0.0840	0.0313	0.0127	0.0006	0.1288
	12	0.1324	0.0431	0.0165	0.0006	0.1929
	16	0.1805	0.0567	0.0210	0.0007	0.2592
YOLO26s-pose	4	0.0876	0.0178	0.0083	0.0005	0.1145
YOLO26m-pose	4	0.2519	0.0177	0.0083	0.0005	0.2786
YOLO26l-pose	4	0.2613	0.0177	0.0083	0.0005	0.2879

Table 1: Average per-frame runtime of the tracking system for different camera and model configurations. Times are given in seconds.

only around 14.3 fps. This is likely insufficient for real-time applications. However, it should be noted that not much effort has gone into optimization of the implementation, and that the test were performed on relatively weak integrated GPU hardware.

5.2 Quantitative Results

Quantitative results for the system on the chosen test sequences can be found in Table 2. When keeping the model fixed to YOLO26n-pose and increasing the number of cameras from 4 to 16, a significant reduction in pose and localization errors can be observed. The joint position error decreases drastically by about half, from 5.439 cm down to 2.576 cm. This indicates that incorporating more viewpoints substantially improves 3D localization and pose estimation accuracy. Also, the increase in camera views brings improvements in recall. Recall increases from 0.975 with 4 cameras to 0.998 when using 16 cameras, indicating fewer missed detections likely because the cameras now cover more angles that would otherwise not be visible. There exists a slight trade-off in tracking accuracy and precision. Adding more cameras leads to a decrease in precision from 0.966 for 4 cameras to 0.955 with 16. This can be explained by the fact that more camera views introduce a higher likelihood of false positives and could also indicate limitations in the association algorithm. MOTA hits a sweet spot of 0.960 with 8 cameras.

In contrast, when keeping the camera count fixed at 4 and scaling up the model complexity from YOLO26n-pose to YOLO26l-pose, we see improvements in all metrics. The largest model achieves the best overall MOTA of 0.960 and precision of 0.977. Upgrading the model capacity also reduces MPJPE from 5.439 cm to 4.554 cm. However, the improvement in this metric is not as significant as with the addition of more cameras.

Finally, Table 3 shows the average performance per scene across performed evaluations. It can be seen that the 160224_haggling1 scene is the one with the lowest performance metrics. In this scene, MOTA and Precision suffer from a large increase in false positives. This scene contains the most entries and exists from the room as well as more close contact, which the tracking algorithm can struggle with. The system performs particularly well on 161029_piano4 and 170915_office1. This is largely because in these scenes the persons are in the room for the entire dura-

tion of the video sequence, avoiding a big source of errors for the system.

5.3 Qualitative Results

Some qualitative evaluation has also been performed for this project.

5.3.1 Impact of the Skeletal Constraints

5.3.2 Common Failure Modes

The results of the system on the tests sequences has been visualized and a number of common issues have been identified. Some examples of these failure modes are shown in Fig. 3.

Partially Occluded Person One very common failure mode is depicted in Fig. 3a. When a person is partially occluded in all camera views, then the position of the unobserved keypoints tends to shift in unnatural ways, as can be seen with the legs keypoints in the given example. This problem is made worse by the fact that the YOLO model tend to predict invisible keypoints in unnatural positions, and while their visibility score is low, it can still commutatively have a significant effect over multiple frames. This issue causes increased joint position errors.

Inactive Track Seen by Only One Camera Another commonly observed failure is shown in Fig. 3b. When a person first enters the room, typically only one camera can see them. This means that it is not possible to create a new track for that person. This causes false positives, and therefore reduced MOTA and Recall, whenever a person enters the room.

Active Track Seen by Only One Camera Figure 3c shows another issue that can occur when only a single camera is able to see a given person. In this case, the track has already been initialized, but the person has moved into a location where they are no longer seen by two cameras. In this case the depth coordinate from the camera can drift as the person moves, causing distorted skeletons before and behind the actual ground truth.

Detection of Person While Exiting Figure 3d show a failure that occurs when a person leaves the room. In this case, the camera that is pointing at the exit will see the person for longer than the actual ground truth indicates.

Model	# Cameras	MOTA	MOTP	MPJPE	Precision	Recall
YOLO26n-pose	4	0.950	5.004	5.439	0.974	0.975
	8	0.960	2.912	2.967	0.966	0.997
	12	0.956	2.795	2.845	0.963	0.998
	16	0.948	2.515	2.576	0.955	0.998
YOLO26s-pose	4	0.953	4.957	5.427	0.974	0.979
YOLO26m-pose	4	0.954	4.745	5.318	0.975	0.980
YOLO26l-pose	4	0.960	4.252	4.554	0.977	0.983

Table 2: Evaluation results for different camera and model configurations. MOTA, MOTP, Precision, and Recall have been computed with an average of 15 centimeters Euclidean distance as the matching threshold. MOTP and MPJPE are given as mean per-joint Euclidean distance in centimeters. The best result in each column is highlighted in bold.

Scene	MOTA	MOTP	MPJPE	Precision	Recall
160224_haggling1	0.862	4.004	5.007	0.900	0.972
161029_build1	0.951	4.086	4.367	0.976	0.975
161029_piano4	0.998	3.395	3.395	0.998	1.000
170915_office1	0.993	4.151	4.178	0.996	0.997
171204_pose6	0.967	3.779	3.858	0.976	0.992

Table 3: Evaluation results for different scenes averaged over all performed evaluations. Evaluation metrics have been calculates as in Table 2.

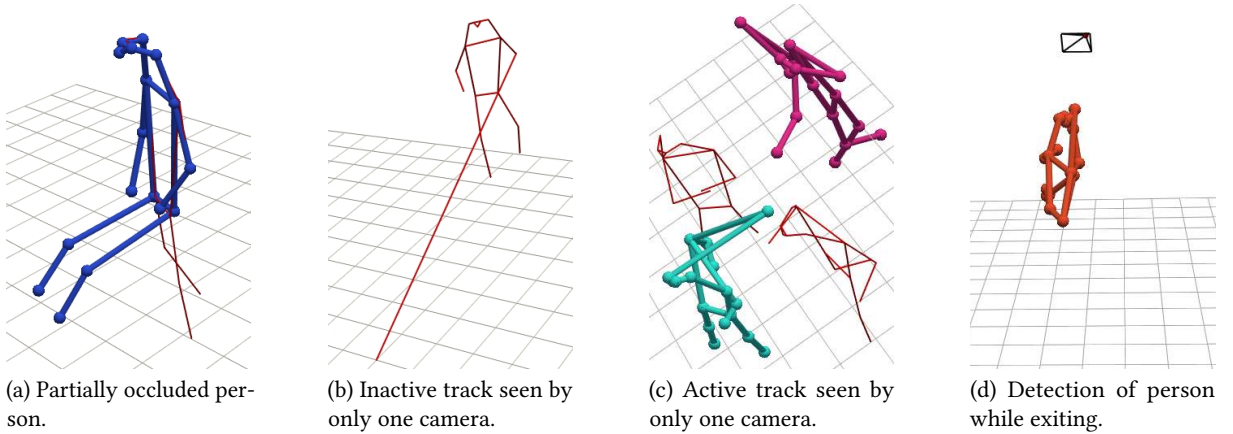


Figure 3: Some qualitative samples extracted from the evaluation results on the test set that exhibit commonly seen issues with the tracking system. Ground truth skeletons are drawn using thin red lines, while predicted skeletons are drawn thicker and with different colors per track.

This causes false positives and therefore worse MOTA and Precision. Note that in this case we also have the above mentioned issue of an active track seen by only one camera.

6 Conclusion and Outlook

In conclusion, in this project a complete end-to-end 3D multi-person tracking pipeline was designed, implemented, and evaluated. The quantitative evaluation on the CMU Panoptic dataset provided some architectural insight. Spatial viewpoint density exhibits a far higher return on tracking accuracy than increasing deep learning model capacity. Scaling the camera view count from 4 to 16 with the YOLO26n-pose model reduced the mean per-joint position error by more than half, from 5.44 cm down to 2.58 cm. In contrast, replacing this lightweight model with the heavy

YOLO26l-pose model under a fixed 4-camera setup yielded smaller accuracy gains at the cost of a massive computational bottleneck, dropping system throughput to approximately 3 FPS.

Despite the relatively high localization accuracy of the system under optimal conditions, there remain several limitations. For example, the process noise matrix, the measurement noise boundaries, the gating thresholds and the physics model were all hand-tuned based on empirical intuition rather than optimized directly from dataset statistics. This restricts the filter’s achievable performance and creates a manual optimization bottleneck. Furthermore, the system models the tracking state of each joint mostly independently. Since the system lacks explicit physical constraints, noisy or partially occluded joint detections can lead to anatomically impossible skeletal deformations. Finally, as demonstrated in the qualitative evaluations, tracking quality degrades

when targets are partially occluded or not in view of the camera views. Since 3D initialization requires triangulation across at least two views, the system exhibits tracking delays during entry and spatial drifting during single-camera visibility.

Possible future improvements to address these limitations could include the learning of system parameters from the dataset. The parameters could be optimized in an end-to-end differentiable manner using backpropagation through time on a set of training sequences. A further improvement could be the addition of anatomical plausibility constraints on the state predictions. These constraints could possibly be included as pseudo-measurements in the update step of the extended Kalman filter. Another avenue of investigation could be the fine-tuning of the YOLO based 2D pose estimation models. Also, the addition of additional information sources, such as monocular depth values could make it possible to track people even using single camera views. Exploring more advanced filtering methods such as particle filters, potentially also using occlusion aware likelihoods could be promising future directions. Finally, the evaluation in this project has been limited by time constraints. As such, the evaluation of the system using a larger set of more diverse sequences would be highly valuable to uncover further areas for improvement.

References

- Ansel, Jason, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, et al. (2024). "Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation". In: *Proceedings of the 29th ACM international conference on architectural support for programming languages and operating systems*, volume 2, pp. 929–947.
- Bar-Shalom, Yaakov, X Rong Li, and Thiagalingam Kirubarajan (2001). *Estimation with applications to tracking and navigation: theory algorithms and software*. John Wiley & Sons.
- Bernardin, Keni and Rainer Stiefelhagen (2008). "Evaluating multiple object tracking performance: the clear mot metrics". In: *EURASIP Journal on Image and Video Processing* 2008.1, p. 246309.
- Bewley, Alex, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft (2016). "Simple online and realtime tracking". In: *2016 IEEE international conference on image processing (ICIP)*. Ieee, pp. 3464–3468.
- Girshick, Ross (2015). "Fast r-cnn". In: *Proceedings of the IEEE international conference on computer vision*, pp. 1440–1448.
- Hartley, Richard and Andrew Zisserman (2003). *Multiple view geometry in computer vision*. Cambridge university press.
- He, Kaiming, Georgia Gkioxari, Piotr Dollár, and Ross Girshick (2017). "Mask r-cnn". In: *Proceedings of the IEEE international conference on computer vision*, pp. 2961–2969.
- Ionescu, Catalin, Dragos Papava, Vlad Olaru, and Cristian Sminchisescu (2013). "Human3.6m: Large scale datasets and predictive methods for 3d human sensing in natural environments". In: *IEEE transactions on pattern analysis and machine intelligence* 36.7, pp. 1325–1339.
- Jocher G. Chaurasia A., Qiu J. (2026). *Ultralytics YOLO*. <https://github.com/ultralytics/ultralytics>. GitHub Repository.
- Joo, Hanbyul, Hao Liu, Lei Tan, Lin Gui, Bart Nabbe, Iain Matthews, Takeo Kanade, Shohei Nobuhara, and Yaser Sheikh (2015). "Panoptic Studio: A Massively Multiview System for Social Motion Capture". In: *The IEEE International Conference on Computer Vision (ICCV)*.
- Joo, Hanbyul, Tomas Simon, Xulong Li, Hao Liu, Lei Tan, Lin Gui, Sean Banerjee, Timothy Scott Godisart, Bart Nabbe, Iain Matthews, Takeo Kanade, Shohei Nobuhara, and Yaser Sheikh (2017). "Panoptic Studio: A Massively Multiview System for Social Interaction Capture". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Kuhn, Harold W (1955). "The Hungarian method for the assignment problem". In: *Naval research logistics quarterly* 2.1-2, pp. 83–97.
- Kurillo, Gregorij, Evan Hemingway, Mu-Lin Cheng, and Louis Cheng (2022). "Evaluating the accuracy of the azure kinect and kinect v2". In: *Sensors* 22.7, p. 2469.
- Lanz, Oswald (Spring 2026). *Computer Vision: Lecture Notes*. Course lecture notes for Free University of Bolzano course number 73075. Accessed June 10, 2026.
- Lin, Tsung-Yi, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick (2014). "Microsoft coco: Common objects in context". In: *European conference on computer vision*. Springer, pp. 740–755.
- Maji, Debapriya, Soyeb Nagori, Manu Mathew, and Deepak Poddar (2022). "Yolo-pose: Enhancing yolo for multi person pose estimation using object keypoint similarity loss". In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 2637–2646.
- Paszke, Adam, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. (2019). "Pytorch: An imperative style, high-performance deep learning library". In: *Advances in neural information processing systems* 32.
- Ren, Shaoqing, Kaiming He, Ross Girshick, and Jian Sun (2015). "Faster r-cnn: Towards real-time object detection with region proposal networks". In: *Advances in neural information processing systems* 28.
- Sapkota, Ranjan, Rahul Harsha Cheppally, Ajay Sharda, and Manoj Karkee (2025). "YOLO26: key architectural enhancements and performance benchmarking for real-time object detection". In: *arXiv preprint arXiv:2509.25164*.
- Sapkota, Ranjan and Manoj Karkee (2025). "Ultralytics YOLO evolution: An overview of YOLO26, YOLO11, YOLOv8 and YOLOv5 object detectors for computer vision and pattern recognition". In: *arXiv preprint arXiv:2510.09653*.

- Simon, Tomas, Hanbyul Joo, and Yaser Sheikh (2017). “Hand Keypoint Detection in Single Images using Multiview Bootstrapping”. In: *CVPR*.
- Virtanen, Pauli, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. (2020). “SciPy 1.0: fundamental algorithms for scientific computing in Python”. In: *Nature methods* 17.3, pp. 261–272.